

SQL PROJECT ON “ONLINE BOOK STORE”

Project Overview

The Online Book Store SQL Project is a database management and data analysis project developed using PostgreSQL. The project simulates a real-world online bookstore system where customers purchase books and place orders online. Designed and analyzed an Online Book Store database project using PostgreSQL with three relational tables: Books, Customers, and Orders. Imported CSV datasets and executed basic to advanced SQL queries for sales analysis, inventory tracking, and many more.

The project consists of three interconnected tables:

- **Books Table** – Stores book-related information such as title, author, genre, price, and stock quantity.
- **Customers Table** – Contains customer details including customer ID, name, email, city, and country.
- **Orders Table** – Maintains transactional records of customer purchases including order date, quantity, and total amount.

The datasets were imported from CSV files into PostgreSQL tables and analyzed using SQL queries to generate meaningful business insights.

Technologies & Tools Used

- **Database:** PostgreSQL
- **Language:** SQL
- **Data Source:** CSV Files

Three Tables created such as –

Books Table, Customers Table, Orders Table

To visualize the data from tables these queries are used -

“SELECT * FROM Books;”

“SELECT * FROM Customers;”

“SELECT * FROM Orders;”

Created by - Swetabh Suman

“SELECT * FROM Books;”

	book_id [PK] integer	title character varying (100)	author character varying (100)	genre character varying (50)	published_year integer	price numeric (10,2)	stock integer
1	1	Configurable modular throughput	Joseph Crane	Biography	1949	21.34	100
2	2	Persevering reciprocal knowledge user	Mario Moore	Fantasy	1971	35.80	19
3	3	Streamlined coherent initiative	Derrick Howard	Non-Fiction	1913	15.75	27
4	4	Customizable 24hour product	Christopher Andrews	Fiction	2020	43.52	8
5	5	Adaptive 5thgeneration encoding	Juan Miller	Fantasy	1956	10.95	16
6	6	Advanced encompassing implementation	Bryan Morgan	Biography	1985	6.56	2
7	7	Open-architected exuding structure	Jacqueline Young	Romance	1927	43.63	95
8	8	Persistent local encoding	Troy Cox	Science Fiction	2019	48.99	84
9	9	Optimized interactive challenge	Colin Buckley	Fantasy	1987	14.33	70
10	10	Ergonomic national hub	Samantha Ruiz	Mystery	2015	24.63	25

“SELECT * FROM Customers;”

	customer_id [PK] integer	name character varying (100)	email character varying (100)	phone character varying (15)	city character varying (50)	country character varying (150)
1	1	Deborah Griffith	balljoseph@wright-keith.net	1234567891	South Craigfort	Denmark
2	2	Crystal Clements	kimberlybennett@curtis.com	1234567892	East Derekberg	Nicaragua
3	3	Susan Fuller	beanmichael@burnett-stewart.net	1234567893	Austinbury	Equatorial Guinea
4	4	Jamie Ramirez	amandahood@warren.com	1234567894	Dianamouth	Slovenia
5	5	Marcus Murphy	connerjohn@yahoo.com	1234567895	Smithbury	Guinea-Bissau
6	6	Stephen Vasquez	ricemiguel@yahoo.com	1234567896	Hamiltonstad	Rwanda
7	7	Susan Hicks	jeffrey91@yahoo.com	1234567897	East Rebecca	Montenegro
8	8	Matthew Johnson	austinkenneth@manning.net	1234567898	Kirstenborough	Israel
9	9	Matthew Williams	jeffrey41@diaz.com	1234567899	Rebeccafurt	Somalia
10	10	Ronald Osborn	staciekelley@heath.com	1234567900	Lake Benjamin	Cameroon

“SELECT * FROM Orders;”

	order_id [PK] integer	customer_id integer	book_id integer	order_date date	quantity integer	total_amount numeric (10,2)
1	1	84	169	2023-05-26	8	188.56
2	2	137	301	2023-01-23	10	216.60
3	3	216	261	2024-05-27	6	85.50
4	4	433	343	2023-11-25	7	301.21
5	5	14	431	2023-07-26	7	136.36
6	6	439	119	2024-10-11	5	249.40
7	7	195	467	2023-10-23	6	82.92
8	8	32	159	2024-05-07	4	144.84
9	9	109	407	2024-01-04	9	379.71
10	10	94	122	2024-07-09	4	123.00

Basic Queries

Q 1. Retrieve all books in 'Fiction' genre.

Query -

```
SELECT * FROM Books
```

```
WHERE genre='Fiction';
```

Output –

	book_id [PK] integer	title character varying (100)	author character varying (100)	genre character varying (50)	published_year integer	price numeric (10,2)	stock integer
1	4	Customizable 24hour product	Christopher Andrews	Fiction	2020	43.52	8
2	22	Multi-layered optimizing migration	Wesley Escobar	Fiction	1908	39.23	78
3	28	Expanded analyzing portal	Lisa Coffey	Fiction	1941	37.51	79
4	29	Quality-focused multi-tasking challenge	Katrina Underwood	Fiction	1905	31.12	100
5	31	Implemented encompassing conglomerat...	Melissa Taylor	Fiction	2010	21.23	44
6	39	Optimized national process improvement	Megan Goodwin	Fiction	1978	10.99	42
7	40	Adaptive didactic interface	Natalie Gonzalez	Fiction	1923	25.97	94
8	47	Reverse-engineered directional conglome...	John Christian	Fiction	2006	20.37	90
9	62	Re-contextualized real-time strategy	Nicole Lynch	Fiction	1953	26.34	23
10	63	Polarized heuristic database	Franklin Mack	Fiction	1989	22.38	56

Q 2. Find the books published after the year 1950.

Query -

```
SELECT * FROM Books
```

```
WHERE published_year>1950;
```

Output -

	book_id [PK] integer	title character varying (100)	author character varying (100)	genre character varying (50)	published_year integer	price numeric (10,2)	stock integer
1	2	Persevering reciprocal knowledge user	Mario Moore	Fantasy	1971	35.80	19
2	4	Customizable 24hour product	Christopher Andrews	Fiction	2020	43.52	8
3	5	Adaptive 5thgeneration encoding	Juan Miller	Fantasy	1956	10.95	16
4	6	Advanced encompassing implementation	Bryan Morgan	Biography	1985	6.56	2
5	8	Persistent local encoding	Troy Cox	Science Fiction	2019	48.99	84
6	9	Optimized interactive challenge	Colin Buckley	Fantasy	1987	14.33	70
7	10	Ergonomic national hub	Samantha Ruiz	Mystery	2015	24.63	25
8	11	Secured zero tolerance time-frame	Denise Barnes	Fantasy	1998	35.95	10
9	12	Polarized optimal array	Destiny Scott	Non-Fiction	1989	27.43	63
10	15	User-friendly motivating strategy	Keith Smith	Non-Fiction	1997	23.83	58

Q 3. List all the customers from the 'Canada'.

Query -

```
SELECT * FROM Customers  
  
WHERE country='Canada';
```

Output -

	customer_id [PK] integer	name character varying (100)	email character varying (100)	phone character varying (15)	city character varying (50)	country character varying (150)
1	38	Nicholas Harris	christine93@perkins.com	1234567928	Davistown	Canada
2	415	James Ramirez	robert54@hall.com	1234568305	Maxwelltown	Canada
3	468	David Hart	stokesrebecca@gmail.c...	1234568358	Thompsonfurt	Canada

Q 4. Show orders placed in November 2023.

Query -

```
SELECT * FROM Orders  
  
WHERE order_date BETWEEN '2023-11-01' AND '2023-11-30';
```

Output -

	order_id [PK] integer	customer_id integer	book_id integer	order_date date	quantity integer	total_amount numeric (10,2)
1	4	433	343	2023-11-25	7	301.21
2	19	496	60	2023-11-17	9	316.26
3	75	291	375	2023-11-30	5	170.75
4	132	469	333	2023-11-22	7	194.32
5	137	474	471	2023-11-25	8	363.04
6	163	207	384	2023-11-23	3	101.76
7	182	129	293	2023-11-01	7	125.51
8	200	313	303	2023-11-23	1	6.57
9	213	325	447	2023-11-17	7	253.75
10	231	22	384	2023-11-11	1	33.92

Q 5. Retrieve the total stock of books available.

Query -

```
SELECT SUM(stock) AS total_stock  
  
FROM Books;
```

Output -

	total_stock bigint
1	25056

Q 6. Find the details of the most expensive book.

Query -

```
SELECT * FROM Books  
  
ORDER BY price DESC LIMIT 1;
```

Output –

	book_id [PK] integer	title character varying (100)	author character varying (100)	genre character varying (50)	published_year integer	price numeric (10,2)	stock integer
1	340	Proactive system-worthy orchestrati...	Robert Scott	Mystery	1907	49.98	88

Q 7. Show all customers who ordered more than 1 quantity of a book.

Query -

```
SELECT * FROM Orders  
  
WHERE quantity>1;
```

Output –

	order_id [PK] integer	customer_id integer	book_id integer	order_date date	quantity integer	total_amount numeric (10,2)
1	1	84	169	2023-05-26	8	188.56
2	2	137	301	2023-01-23	10	216.60
3	3	216	261	2024-05-27	6	85.50
4	4	433	343	2023-11-25	7	301.21
5	5	14	431	2023-07-26	7	136.36
6	6	439	119	2024-10-11	5	249.40
7	7	195	467	2023-10-23	6	82.92
8	8	32	159	2024-05-07	4	144.84
9	9	109	407	2024-01-04	9	379.71
10	10	94	122	2024-07-09	4	123.00

Q 8. Retriveve all orders where the total amount exceeds \$20.

Query -

```
SELECT * FROM Orders  
  
WHERE total_amount>20;
```

Output –

	order_id [PK] integer	customer_id integer	book_id integer	order_date date	quantity integer	total_amount numeric (10,2)
1	1	84	169	2023-05-26	8	188.56
2	2	137	301	2023-01-23	10	216.60
3	3	216	261	2024-05-27	6	85.50
4	4	433	343	2023-11-25	7	301.21
5	5	14	431	2023-07-26	7	136.36
6	6	439	119	2024-10-11	5	249.40
7	7	195	467	2023-10-23	6	82.92
8	8	32	159	2024-05-07	4	144.84
9	9	109	407	2024-01-04	9	379.71
10	10	94	122	2024-07-09	4	123.00

Q 9. List all genres available in the books table.

Query -

```
SELECT DISTINCT genre
FROM Books;
```

Output –

	genre character varying (50)
1	Romance
2	Biography
3	Mystery
4	Fantasy
5	Fiction
6	Non-Fiction
7	Science Fiction

Q 10. Find the book with lowest stock.

Query -

```
SELECT * FROM Books
ORDER BY stock LIMIT 1;
```

Output –

	book_id [PK] integer	title character varying (100)	author character varying (100)	genre character varying (50)	published_year integer	price numeric (10,2)	stock integer
1	127	Business-focused real-time benchm...	David Nelson	Science Fiction	1997	11.66	0
2	44	Networked systemic implementation	Ryan Frank	Science Fiction	1965	13.55	0
3	60	Robust eco-centric capacity	Brian Haney	Biography	1990	35.14	0
4	163	Object-based eco-centric challenge	Douglas Mccarthy	Non-Fiction	1905	19.11	0

Q 11. Calculate the total revenue generated from all orders.

Query -

```
SELECT SUM(total_amount) AS Total_revenue  
FROM orders;
```

Output –

	total_revenue numeric
1	75628.66

Advanced Queries

Q 1. Retrieve the total number of books sold for each genre.

Query -

```
SELECT b.genre, SUM(o.quantity) AS total_books_sold  
FROM Orders o  
JOIN Books b  
ON o.book_id=b.book_id  
GROUP BY b.genre;
```

Output –

	genre character varying (50)	total_books_sold bigint
1	Romance	439
2	Biography	285
3	Mystery	504
4	Fantasy	446
5	Fiction	225
6	Non-Fiction	351
7	Science Fiction	447

Q 2. Find the average price of books in the 'Fantasy' genre.

Query -

```
SELECT AVG(price) AS Average_price  
FROM Books
```

WHERE genre='Fantasy';

Output –

	average_price numeric
1	25.98

Q 3. List customers who have placed at least 2 orders

Query -

```
SELECT o.customer_id, c.name, COUNT(o.order_id) AS Order_count
FROM Orders o
JOIN Customers c
ON o.customer_id=c.customer_id
GROUP BY o.customer_id, c.name
HAVING COUNT(order_id) >=2;
```

Output –

	customer_id integer	name character varying (100)	order_count bigint
1	225	Christopher Mccullough	2
2	418	Kiara Blankenship MD	3
3	322	William Cameron	3
4	325	Emily Vargas	4
5	376	Justin Donaldson	2
6	486	Melanie Kelly	2
7	461	Crystal Pierce	3
8	2	Crystal Clements	2
9	149	Jason Robinson	3
10	173	Victoria Dixon	2

Q 4. Find the most frequently ordered book.

Query -

```
SELECT o.book_id, b.title, COUNT(o.order_id) AS Order_Count
FROM Orders o
JOIN Books b
```

```

ON o.book_id=b.book_id

GROUP BY o.book_id, b.title

ORDER BY Order_Count DESC LIMIT 1;

```

Output –

	book_id integer	title character varying (100)	order_count bigint
1	88	Robust tangible hardwa...	4

Q 5. Show the top 3 most expensive books of 'Fantasy' genre.

Query -

```

SELECT * FROM Books

WHERE genre='Fantasy'

ORDER BY price DESC

LIMIT 3;

```

Output –

	book_id [PK] integer	title character varying (100)	author character varying (100)	genre character varying (50)	published_year integer	price numeric (10,2)	stock integer
1	240	Stand-alone content-based hub	Lisa Ellis	Fantasy	1957	49.90	41
2	462	Innovative 3rdgeneration databa...	Allison Contreras	Fantasy	1988	49.23	62
3	238	Optimized even-keeled analyzer	Sherri Griffith	Fantasy	1975	48.97	72

Q 6. retrieve the total quantity of books sold by each author.

Query -

```

SELECT b.author, SUM(o.quantity) AS Total_Quantity

FROM Orders o

JOIN Books b

ON o.book_id=b.book_id

GROUP BY b.author;

```

Output –

	author character varying (100) 🔒	title character varying (100) 🔒	total_books_sold bigint 🔒
1	Amanda Wilson	Multi-tiered responsive parallelism	20
2	Andrea Eaton	Networked contextually-based throughput	10
3	Angela Richard	Object-based secondary emulation	10
4	Anthony Nguyen	Enhanced next generation standardization	10
5	Brad Vasquez	Innovative empowering concept	10
6	Brian Baker	Exclusive 24hour groupware	10
7	Casey Moreno	Multi-layered 24hour infrastructure	10
8	Cory Ross	Optimized attitude-oriented synergy	10
9	Daniel Gillespie	Intuitive 4thgeneration intranet	10
10	Ellen Doyle	Synergized intangible methodology	10

Q 7. List the cities where customers who spent over \$30 are located.

Query -

```

SELECT DISTINCT(c.city), total_amount
FROM Orders o
JOIN Customers c
ON o.customer_id=c.customer_id
WHERE o.total_amount>30;

```

Output –

	city character varying (50) 🔒	name character varying (100) 🔒	total_amount numeric (10,2) 🔒
1	Smithborough	Christine Maldonado	491.50
2	Fieldsland	Jason Villegas	489.60
3	Thomaschester	Kevin Collins	489.60
4	Mendezburgh	Catherine Wilkins	486.70
5	Melissaside	Annette Garcia	480.30
6	Freemanland	Julie Smith	469.70
7	Bakerton	Jonathon Strickland	469.30
8	West Kimberlyhaven	Laura Duarte	466.60
9	Elizabethshire	Ashley Perez	465.40
10	Lake Charleshaven	Robert Clark	459.10

Q 8. Find the customers who spent the most on orders.

Query -

```
SELECT c.customer_id, c.name, SUM(o.total_amount) AS Total_spent
FROM Orders o
JOIN Customers c
ON o.customer_id=c.customer_id
GROUP BY c.customer_id
ORDER BY Total_spent DESC;
```

Output –

	customer_id [PK] integer	name character varying (100)	city character varying (50)	total_spent numeric
1	457	Kim Turner	South Rachelview	1398.90
2	174	Jonathon Strickland	Bakerton	1080.95
3	364	Carrie Perez	Kennethland	1052.27
4	405	Julie Smith	Freemanland	991.00
5	386	Pamela Gordon	East Richardburgh	986.30
6	425	Ashley Perez	Elizabethshire	942.62
7	474	Anthony Young	East Chelsea	929.19
8	163	Robert Clark	Lake Charleshaven	746.65
9	167	Justin Spencer	Christopherchester	719.93
10	214	Alexander Scott	Matthewfurt	682.15

Q 9. Calculate the stock remaining after fulfilling all orders.

Query -

```
SELECT b.book_id, b.Title, b.stock, COALESCE(SUM(o.quantity), 0) AS
order_quantity,
       b.stock-COALESCE(SUM(o.quantity),0) AS remaining_quantity
FROM Books b
LEFT JOIN Orders o
ON b.book_id=o.book_id
GROUP BY b.book_id
```

ORDER BY b.book_id;

Output –

	book_id [PK] integer 	title character varying (100) 	stock integer 	order_quantity bigint 	remaining_quantity bigint 
1	1	Configurable modular throughput	100	3	97
2	2	Persevering reciprocal knowledge user	19	0	19
3	3	Streamlined coherent initiative	27	5	22
4	4	Customizable 24hour product	8	0	8
5	5	Adaptive 5thgeneration encoding	16	8	8
6	6	Advanced encompassing implementation	2	0	2
7	7	Open-architected exuding structure	95	5	90
8	8	Persistent local encoding	84	3	81
9	9	Optimized interactive challenge	70	0	70
10	10	Ergonomic national hub	25	1	24